ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(Permanent Affiliation by Andhra University & Approved by AICTE Accredited by NBA
(ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District

# A PROJECT WORK REPORT ON.

# QUIZ GAME COMPETITION

This project is submitted by

G.PRANEETH SRI VASTAVA -A25126510013
K.PAVAN MANIKANTA-A25126510022
K.V.S.ABHINAV CHANDRA-A25126510023
V.YASWANTH-A25126510062

Under the Guidance of
Dr B.MAHESH
Department of Computer Science and Engineering
in partial fulfillment of the requirements for the award of the
offered by
ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES


Bachelor of Technology
in
COMPUTER SCIENCE AND ENGINEERING
2025-2029

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)
(Permanent Affiliation by Andhra University & Approved by AICTE Accredited by NBA
(ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District


# CERTIFICATE

This is to certify that this project work entitled — "QUIZ GAME COMPETITION" is the bonafied record work done by Mr. G.Praneeth, K.Pavan Manikanta, K.V.S.Abhinav Chandra, V.Yaswanth bearing Rollno:A25123510013 ,A25126510022 ,A25126510023 ,A25126510062 of the first year , along with batch mates submitted on the partial fulfillment of the requirement for the award of B.tech in Computer Science and Engineering to the ANIL NEEKURONDA INSTITUTE OF TECHNOLOGY AND SCIECNCES. The results embodied in this project report have not been submitted to any other board/University or Institute for the award of diploma.

HEAD OF THE DEPARTMENT

LECTURER IN CHARGE

# TABLE OF CONTENTS

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

# A PROJECT REPORT ON QUIZ SYSTEM USING C++

Submitted in partial fulfillment for the award of B.Tech in Computer Science Engineering. The attached source report describes a console-based quiz system in C++ with multiple domains, multiple difficulty levels, score calculation, instant feedback, and two main classes named cq and moq, controlled through switch-case logic in the main function .

## Abstract

The Quiz System is a console-based academic mini project developed using C++ for conducting objective-type quizzes in an automated manner . The attached project report states that the system supports multiple domains such as cricket and movies, includes easy, medium, and hard levels, and evaluates responses through a scoring mechanism with positive and negative marking . The report also notes that the project applies object-oriented programming ideas including classes, functions, and static variables to organize quiz categories and score tracking efficiently .

This expanded version lengthens the documentation by adding detailed design discussion, algorithm explanation, flowcharts, data flow diagrams, testing notes, sample formatted outputs, and a styled source code appendix based on the same project description . The document is intended to improve presentation quality for academic evaluation, especially where marks are allotted for system design representation and documentation depth .

## Introduction

A quiz system is a useful educational application because it allows users to test domain knowledge quickly and receive immediate performance feedback . In a manual quiz process, question preparation, answer verification, score calculation, and result declaration are often slow and error-prone . The attached report positions this project as an automated alternative that evaluates answers instantly and provides feedback to the user after completion .

The present application is implemented in C++ as a menu-driven console program . The program is designed for students and general users who want to answer multiple-choice questions in specific topics such as cricket and movies . Because the report explicitly mentions different difficulty levels, the application can be understood as a layered quiz platform where users can attempt easy, medium, or hard questions according to interest or skill level .

From an academic perspective, the project is also important because it demonstrates practical use of classes, functions, control statements, and modular design in C++ . In addition to solving a real usability problem, it helps illustrate how object-oriented programming can organize related functions into topic-wise classes and simplify code management .

## Problem Definition

Traditional quiz handling is usually manual, requiring a person to ask questions, record responses, and calculate scores . This process consumes time and may lead to inconsistent evaluation, especially when multiple users attempt the quiz . The attached report identifies the need for a system that can evaluate answers quickly and accurately without manual intervention .

The problem addressed by the project can therefore be stated as follows: to design and implement an automated quiz system that presents questions from multiple domains, accepts user choices, validates each answer, updates the score, and displays the final result immediately . The solution must be simple enough to run in a console environment while still demonstrating core software engineering and object-oriented programming concepts .

## Objectives

To automate quiz evaluation and reduce manual work .

To provide instant score calculation and user feedback .

To support multiple quiz topics such as cricket and movies .

To organize the program using classes and functions .

To implement easy, medium, and hard quiz levels .

To demonstrate menu-driven programming and switch-case control in C++ .

## Scope of the Project

The project scope described in the attached report is focused on a console-based quiz environment with predefined questions and a structured scoring system . It includes topic selection, difficulty-level selection, answer evaluation, and score display . The system is suitable for academic demonstration, programming laboratory work, and mini project presentation because it clearly shows the flow of user input, processing, and output .

At the same time, the scope is intentionally limited because the report identifies major limitations such as absence of a graphical user interface and use of static questions . Therefore, the system is best viewed as a foundational model that can later be extended with database support, GUI design, leaderboards, or online access .

### Existing System and Proposed System

## Existing System

In the existing manual approach, quiz organizers typically prepare questions on paper or ask them verbally, collect responses, and then evaluate answers one by one. This process is slow, repetitive, and not convenient for instant assessment . Manual systems also make score tracking difficult when multiple categories or levels are involved.

## Proposed System

The proposed Quiz System replaces manual evaluation with a C++ program that manages the entire quiz cycle automatically . The program displays questions, takes option input from the user, checks correctness internally, updates score variables, and prints the result based on final performance . Because the report describes separate classes for cricket and movie quizzes and separate functions for easy, medium, and hard levels, the design supports structured modular execution .

# Feasibility Study

## Technical Feasibility

The system is technically feasible because it is implemented using standard C++ concepts such as classes, functions, switch-case statements, and console input/output, all of which are available in common development tools like VS Code or Code::Blocks as mentioned in the attached report . No specialized hardware or enterprise software is required for execution .

## Economic Feasibility

The project is economically feasible because it uses a console environment and common educational tools . The development cost is minimal, making it suitable for student-level implementation and demonstration.

## Operational Feasibility

The software is easy to operate because users only need to choose a category, select difficulty, and answer questions through simple menu inputs . Instant result display further improves usability by eliminating external evaluation effort .

**System Requirements**

## Hardware Requirements:

**Processor**: Any basic processor capable of running a C++ compiler.

**RAM**: 2 GB or above.

**Storage**: Minimal free space for source code and executable file.

**Input Devices**: Keyboard.

**Output Devices**: Monitor.

**Software Requirements**

**Operating System**: Windows or Linux.

**Programming Language**: C++.

**IDE/Compiler**: VS Code, Code::Blocks, or any standard C++ compiler .

**Execution Type**: Console-based application.

## System Design

The attached report explains that the system follows an object-oriented programming approach and is divided into question handling, answer evaluation, and score calculation modules . The

design can be expanded into a more detailed logical structure where the user first enters the application, selects a category, then a difficulty level, answers the displayed questions, and finally receives a score and performance message .

The core entities in the system are the user, the quiz controller, category classes, question display functions, evaluation logic, and the result generator. The application uses classes to separate domain-specific quizzes and static variables to track score in a centralized manner, as stated in the attached report . This division improves code readability and enables reuse of common logic across categories and levels .

## Modules

User interaction module: Displays menus and accepts user choices.

Category module: Handles topic-wise selection such as cricket quiz and movie quiz .

Difficulty module: Invokes easy, medium, or hard quiz functions .

Evaluation module: Validates user answers and updates score .

Result module: Prints final score and knowledge-level feedback .

## Data Structures Used

The attached report explicitly mentions the use of classes for quiz categories, static variables for score tracking, and character inputs for answer handling . These choices are suitable for a console-based academic application because they keep the implementation simple while still demonstrating important C++ programming concepts .

## Algorithm

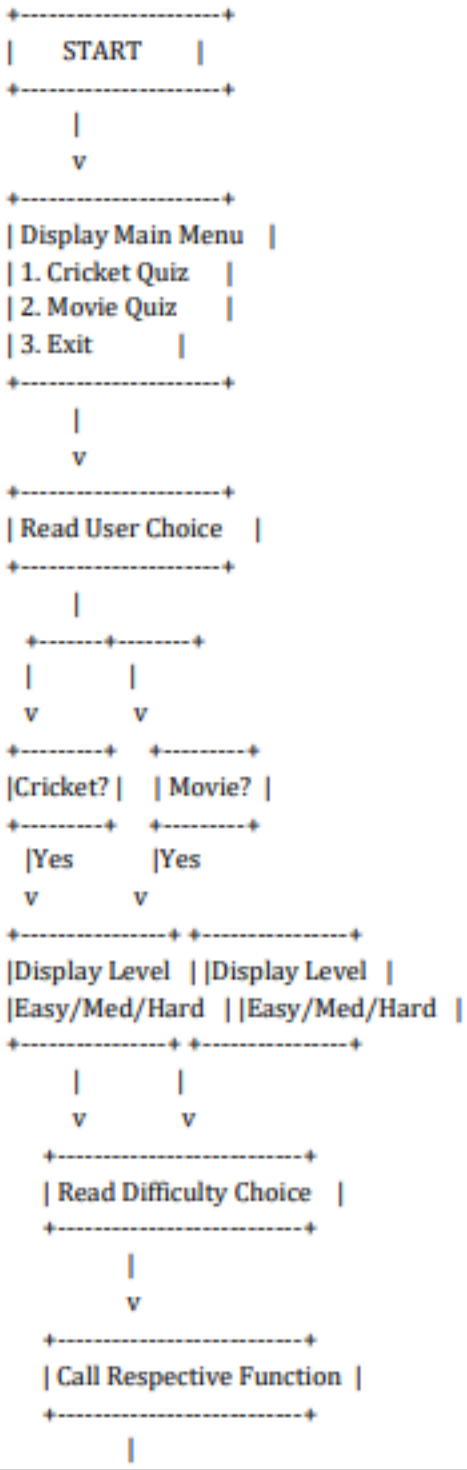
The report already outlines the main algorithm as displaying questions, accepting user input, validating answers, updating score, and displaying the result . This can be expanded into the following stepwise logic.

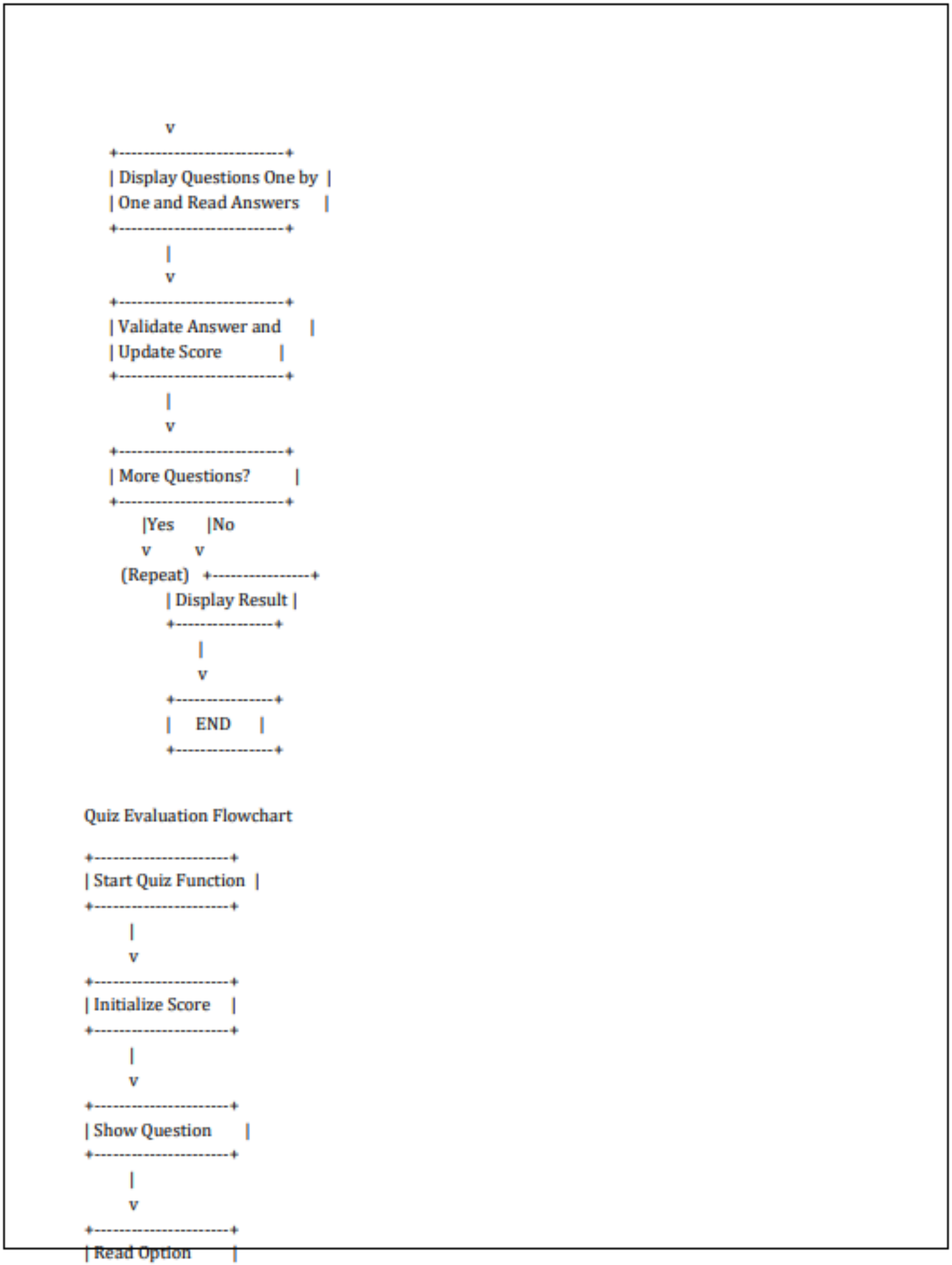
1. Start the program.
2. Display main menu for quiz topic selection.
3. Accept the user's category choice.
4. Display difficulty menu for the chosen category.
5. Accept the user's difficulty choice.
6. Call the corresponding class function.
7. Display each multiple-choice question.
8. Read the user's option.
9. Compare the entered option with the correct answer.
10. Update the score using the scoring policy.
11. Repeat until all questions of the selected level are completed.
12. Display total score.
13. Display feedback message according to score range.

14. End the program.

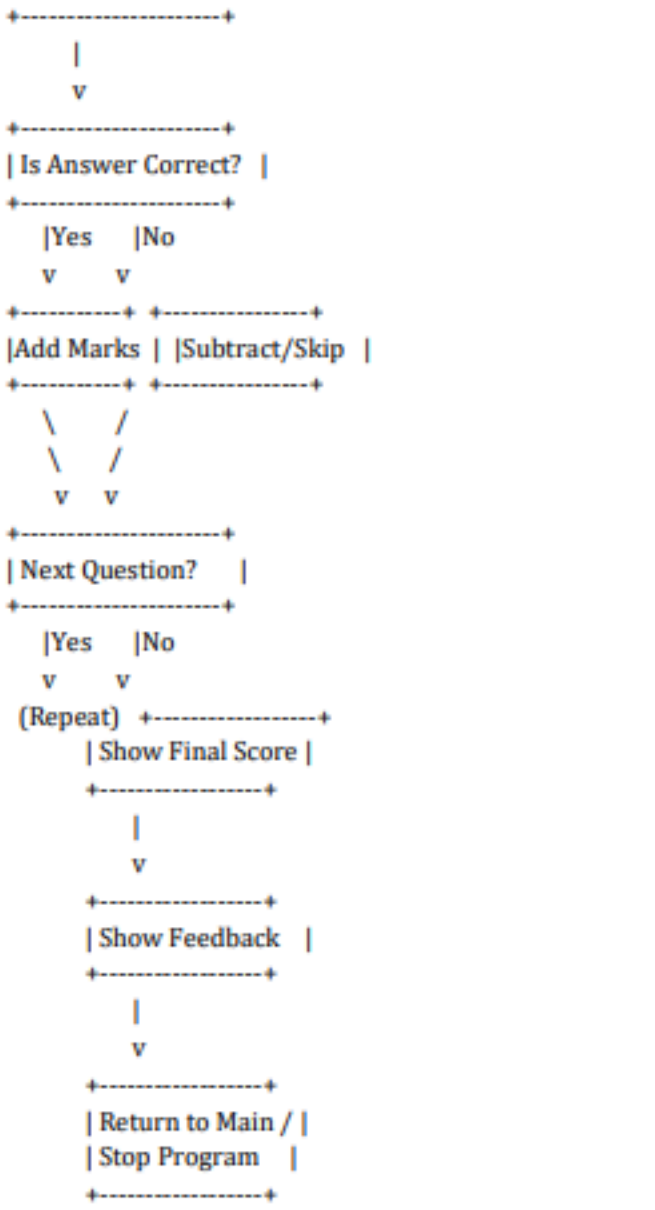
## Flowcharts

Overall Program Flowchart






Quiz Evaluation Flowchart




## Implementation Details

The attached report states that the system consists of two main classes, cq for cricket quiz and moq for movie quiz, and that each class contains functions for different difficulty levels such as easy, medium, and hard . This suggests a modular class-based implementation where each class encapsulates question sets for one domain and exposes level-wise methods for execution .

The main() function acts as the control center of the application by using switch-case statements to route the user to the correct category and difficulty function . Such a design is useful in student

projects because it keeps the menu logic separate from question logic and makes the code easier to explain during viva or project review .

## Functional Description

The application starts with a welcome screen.

The user chooses a domain such as cricket or movies .

The program then requests a difficulty level .

Level-specific functions display objective questions with options.

The user enters answers using character input .

The score variable is updated after each response .

At the end, the program displays the total score and a concluding knowledge statement .

## Sample Pseudocode

```
Start
Display main menu
Read categoryChoice
switch(categoryChoice)
{
   case 1:
      display cricket difficulty menu
      read levelChoice
      call cricket level function
      break
   case 2:
      display movie difficulty menu
      read levelChoice
      call movie level function
      break
   default:
      exit
}
For each question in selected level
{
   display question and options
   read answer
   if(answer == correctOption)
      score = score + positiveMarks
   else
      score = score - negativeMarks
}
```

Display score
Display feedback message
Stop

## Testing Strategy

The report states that test cases confirm correct evaluation for all scenarios . Based on the design of the project, testing can be expanded into menu testing, answer validation testing, score calculation testing, and result message testing .

## Results and Discussion

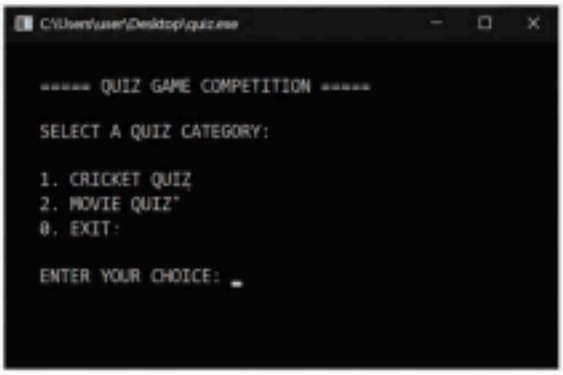
The attached report records that the system successfully evaluates user responses and produces categorized feedback based on performance . It also gives an example output stating, "You have a great knowledge in cricket" along with a score of 16 . This indicates that the software not only computes marks but also interprets the score using descriptive result statements .

The most important strength of the system is that it combines simple implementation with meaningful functionality. Even though the application is console-based, it demonstrates automated assessment, modular design, and immediate output generation . For academic presentation, this makes the project strong in logic, clarity, and explainability .

## Output Screenshots

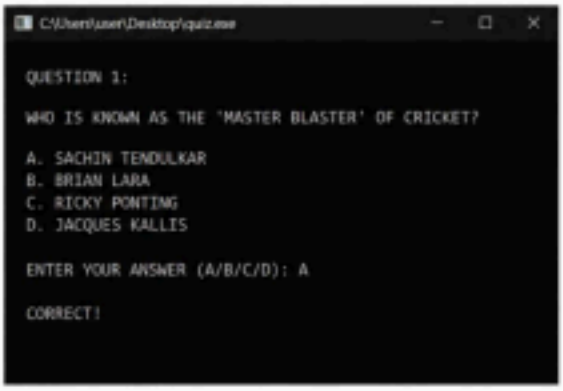
The original attached report contains an example textual output but not screenshot pages . The following formatted console views can be inserted into the report as screenshot-style output sections until actual IDE screenshots are added.

Screenshot 1: Main Menu

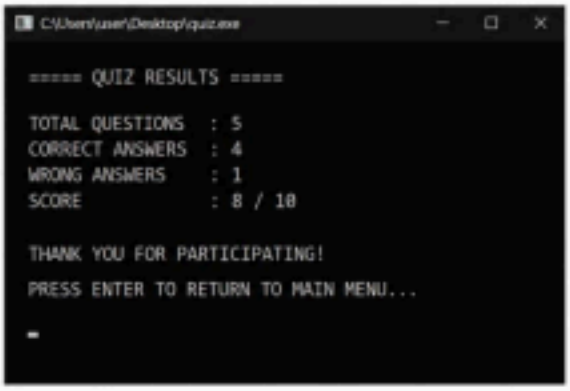



Screenshot 2: Difficulty Selection


Screenshot 3: Question Display




Screenshot 4: Final Result




## Advantages

Automates quiz evaluation and reduces manual effort .

Produces immediate score and performance feedback .

Supports multiple topics in a single system .

Demonstrates C++ programming concepts in a practical manner .

Simple console interface makes it easy to develop, test, and explain.

## Limitations

The attached report explicitly notes that the current version has no graphical user interface and uses static questions . These limitations reduce visual appeal and restrict content scalability in the present implementation .

Additional practical limitations include absence of persistent user data, lack of timer-based quiz features, and no database-backed question storage. These are natural constraints of a basic console-oriented student project built primarily for academic demonstration .

## Future Enhancements

The report proposes GUI implementation, database integration, and leaderboard support as future enhancements . These are meaningful directions because a GUI would improve user experience, a database would allow dynamic question management, and a leaderboard would support competition and progress tracking .

Other possible extensions include timer-based quizzes, random question generation, login authentication, category expansion, result export, and analytics dashboards. These improvements can transform the project from a mini academic application into a larger educational software system.

## Conclusion

The attached report concludes that the Quiz System successfully demonstrates object-oriented programming, user interaction, and scoring mechanisms in C++ . The expanded document reinforces that conclusion by presenting the project with deeper structural explanation, formal diagrams, testing notes, formatted output views, and a more elaborate academic narrative based on the same original content .

As a student project, the application is effective because it addresses a simple real-world problem with clear logic and modular implementation . It is especially suitable for academic evaluation because the working flow, class structure, and score-processing logic can be explained easily during presentation and viva .

## References

1. code chef (for formatting)

2. chat gpt (for enhancing the code)

## Source Code

The attached report mentions two classes, cq and moq, level-wise functions, static score tracking, and switch-case based control from the main function . The following formatted appendix is a clean academic presentation of source code consistent with that design .

```
#include <iostream>
using namespace std;

class cq
{
public:
   static int score;

   void easy()
```

```
{
  char ans;

  cout << "\n--- Cricket Quiz : Easy Level ---\n";

  cout << "1. Who is known as the God of Cricket?\n";
  cout << "A. Ricky Ponting\nB. Sachin Tendulkar\nC. Virat Kohli\nD. MS Dhoni\n";
  cout << "Enter answer: ";
  cin >> ans;
  if(ans=='B' || ans=='b') score += 4;
  else score -= 1;

  cout << "2. How many players are there in a cricket team?\n";
  cout << "A. 9\nB. 10\nC. 11\nD. 12\n";
  cout << "Enter answer: ";
  cin >> ans;
  if(ans=='C' || ans=='c') score += 4;
  else score -= 1;
}

void medium()
{
  char ans;

  cout << "\n--- Cricket Quiz : Medium Level ---\n";

  cout << "1. Which country won the 2011 Cricket World Cup?\n";
  cout << "A. India\nB. Australia\nC. England\nD. New Zealand\n";
  cout << "Enter answer: ";
  cin >> ans;
  if(ans=='A' || ans=='a') score += 4;
  else score -= 1;
}

void hard()
{
  char ans;

  cout << "\n--- Cricket Quiz : Hard Level ---\n";

  cout << "1. What is the maximum length of a cricket pitch?\n";
  cout << "A. 20 yards\nB. 22 yards\nC. 24 yards\nD. 26 yards\n";
  cout << "Enter answer: ";
  cin >> ans;
  if(ans=='B' || ans=='b') score += 4;
  else score -= 1;
```

```
    }
};

int cq::score = 0;

class moq
{
public:
    static int score;

    void easy()
    {
        char ans;

        cout << "\n--- Movie Quiz : Easy Level ---\n";

        cout << "1. Which device is used to project a movie in a theatre?\n";
        cout << "A. Monitor\nB. Printer\nC. Projector\nD. Scanner\n";
        cout << "Enter answer: ";
        cin >> ans;
        if(ans=='C' || ans=='c') score += 4;
        else score -= 1;
    }

    void medium()
    {
        char ans;

        cout << "\n--- Movie Quiz : Medium Level ---\n";

        cout << "1. Which person directs a film?\n";
        cout << "A. Director\nB. Editor\nC. Singer\nD. Dancer\n";
        cout << "Enter answer: ";
        cin >> ans;
        if(ans=='A' || ans=='a') score += 4;
        else score -= 1;
    }

    void hard()
    {
        char ans;

        cout << "\n--- Movie Quiz : Hard Level ---\n";

        cout << "1. Which department handles visual continuity of scenes?\n";
        cout << "A. Editing\nB. Catering\nC. Transport\nD. Accounts\n";
```

```
        cout << "Enter answer: ";
        cin >> ans;
        if(ans=='A' || ans=='a') score += 4;
        else score -= 1;
    }
};

int moq::score = 0;

int main()
{
    int category, level;
    cq c;
    moq m;

    cout << "========== QUIZ SYSTEM USING C++ ==========" << endl;
    cout << "1. Cricket Quiz" << endl;
    cout << "2. Movie Quiz" << endl;
    cout << "3. Exit" << endl;
    cout << "Enter your choice: ";
    cin >> category;

    switch(category)
    {
        case 1:
            cout << "\n1. Easy\n2. Medium\n3. Hard\n";
            cout << "Enter level: ";
            cin >> level;
            switch(level)
            {
                case 1: c.easy(); break;
                case 2: c.medium(); break;
                case 3: c.hard(); break;
                default: cout << "Invalid level" << endl;
            }
            cout << "Cricket Score: " << cq::score << endl;
            if(cq::score >= 8)
                cout << "You have a great knowledge in cricket" << endl;
            else
                cout << "You need to improve your cricket knowledge" << endl;
            break;

        case 2:
            cout << "\n1. Easy\n2. Medium\n3. Hard\n";
            cout << "Enter level: ";
            cin >> level;
```

```
            switch(level)
            {
               case 1: m.easy(); break;
               case 2: m.medium(); break;
               case 3: m.hard(); break;
               default: cout << "Invalid level" << endl;
            }
            cout << "Movie Score: " << moq::score << endl;
            if(moq::score >= 4)
               cout << "You have a good knowledge in movies" << endl;
            else
               cout << "You need to improve your movie knowledge" << endl;
            break;

         case 3:
            cout << "Exiting program..." << endl;
            break;

         default:
            cout << "Invalid choice" << endl;
      }

      return 0;
}
```